

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: ITA 0601

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO4: K- Nearest Neighbour Learning – Locally weighted Regression – Radial Bases Functions – Case Based Learning **(BL 5)**

Assessment Tool Weightage: 20%

QUESTIONS: 15

Total Marks:25

Annexure A

Industry Problem Solving

Code of Conduct:

I N.HarshaVardhan(192324189)certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: N.HarshaVardhan

Annexure A

Short Answer Test

1, A retail company wants to recommend products to customers based on past purchases. Design a solution using K-Nearest Neighbour or Case-Based Learning. Explain how similarity is measured, how recommendations are generated, and how the system adapts to new customer data.

Answer:

Product Recommendation Using K-Nearest Neighbour (KNN)

Objective

The retail company can use **K-Nearest Neighbour (KNN)** to recommend products by finding customers who have similar purchase patterns and recommending products that those similar customers have purchased.

Input Data

The system can maintain customer purchase information such as:

Customer	Laptop	Mouse	Keyboard	Headphones	Printer
----------	--------	-------	----------	------------	---------

C1	1	1	1	0	0
C2	1	1	0	1	0
C3	0	1	1	1	1
C4	1	0	1	1	0
C5	0	1	0	1	1

Here, 1 means the customer purchased the product and 0 means they did not.

Similarity Measurement

For numerical purchase features, **Euclidean distance** can be used:

A smaller distance means that two customers have more similar purchasing behaviour.

For example, if a new customer has purchased:

[1, 1, 0, 0, 0]

the system calculates the distance between this customer and every existing customer. The **K closest customers** are selected.

For purchase/binary data, measures such as **Jaccard similarity** can also be useful because they focus on products that both customers purchased.

Recommendation Generation

The recommendation process is:

1. Collect the new customer's purchase history.
2. Represent the purchase history as a feature vector.
3. Calculate similarity/distance with existing customers.
4. Select the **K most similar customers**.
5. Find products purchased by these neighbours but not purchased by the new customer.
6. Rank the products according to how frequently they appear among the neighbours.
7. Recommend the highest-ranked products.

Example:

Suppose the three nearest customers purchased:

- Customer 1 → Mouse, Keyboard
- Customer 2 → Mouse, Headphones
- Customer 3 → Mouse, Keyboard, Headphones

The system can recommend:

1. **Mouse** – purchased by 3 similar customers
2. **Keyboard** – purchased by 2 similar customers
3. **Headphones** – purchased by 2 similar customers

Adapting to New Customer Data

KNN is a **lazy learning algorithm**, so it does not need to build a complex model during training. When a new customer's purchase is recorded, the new data can simply be added to the customer dataset.

For example:

Old customers → C1, C2, C3, C4, C5

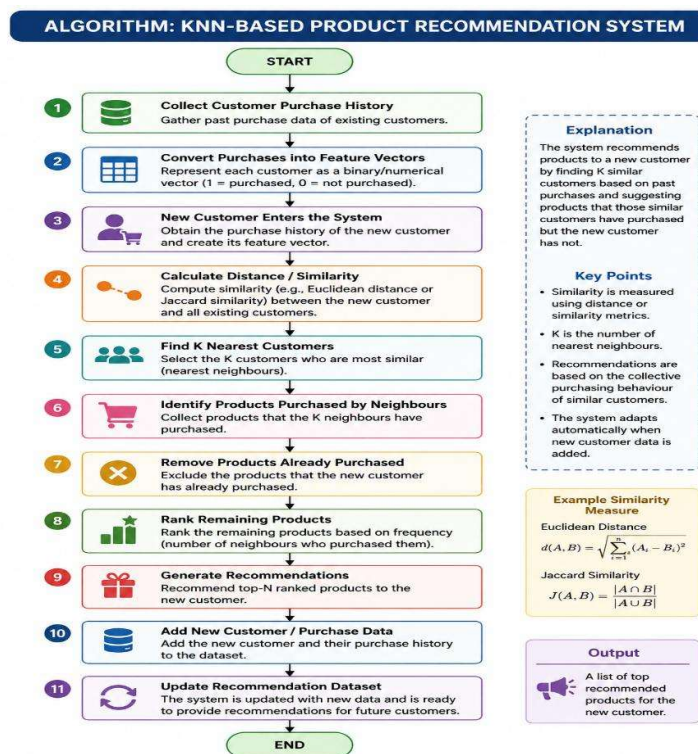
New customer → C6

Purchase → Laptop, Mouse

Updated dataset → C1, C2, C3, C4, C5, C6

When C6 makes another purchase, the system recalculates recommendations using the updated purchase history.

Overall Algorithm



Conclusion

A KNN-based recommendation system can effectively recommend retail products by comparing a customer's purchase history with other customers. Similarity is calculated using distance or similarity measures, and products purchased by the nearest customers are recommended. As new purchases are added, the customer dataset grows, allowing the system to continuously adapt and provide more relevant recommendations.

2. A manufacturing industry needs to predict machine output based on sensor readings that vary over time. Propose a solution using Locally Weighted Regression. Explain how local models improve prediction accuracy and how the system handles noisy or incomplete data.

Answer:

Locally Weighted Regression for Machine Output Prediction

A manufacturing industry can use **Locally Weighted Regression (LWR)** to predict machine output from sensor readings such as temperature, pressure, vibration, speed, and humidity. LWR is useful when the relationship between sensor values and machine output changes over time or is not globally linear.

Proposed Solution

The system collects sensor readings continuously:

Sensors → Data Preprocessing → LWR Model → Predicted Machine Output → Monitoring/Alert

For a new sensor reading, LWR gives higher importance to historical observations that are **similar or close** to the new reading and lower importance to distant observations.

How Local Models Improve Accuracy

Unlike ordinary linear regression, which creates one model for the entire dataset, LWR creates a **local model around each prediction point**.

For example, suppose the machine output depends on temperature:

- At low temperatures, output may increase rapidly with temperature.
- At medium temperatures, output may increase slowly.
- At high temperatures, output may decrease because of overheating.

A single global regression line may not represent all these behaviors accurately. LWR instead concentrates on nearby observations and produces a prediction based on the **local behavior** of the machine.

The weight of each training observation can be calculated using a Gaussian function: where:

- = current sensor reading
- = historical sensor reading
- = distance between the readings
- = bandwidth controlling how quickly the weight decreases
- = importance of the historical observation

Thus, **nearby data receives a high weight**, while distant data receives a low weight.

Handling Noisy Data

Sensor measurements often contain noise caused by vibration, electrical interference, or temporary sensor errors. LWR reduces the effect of noisy observations because:

1. The model considers multiple nearby observations rather than relying on a single reading.
2. Distant abnormal observations receive very small weights.
3. A preprocessing stage can remove extreme outliers.
4. Moving averages or median filtering can be applied to sensor readings before prediction.

For example, if the temperature sensor suddenly reports an unrealistic value, preprocessing can identify it as an outlier before it significantly affects the prediction.

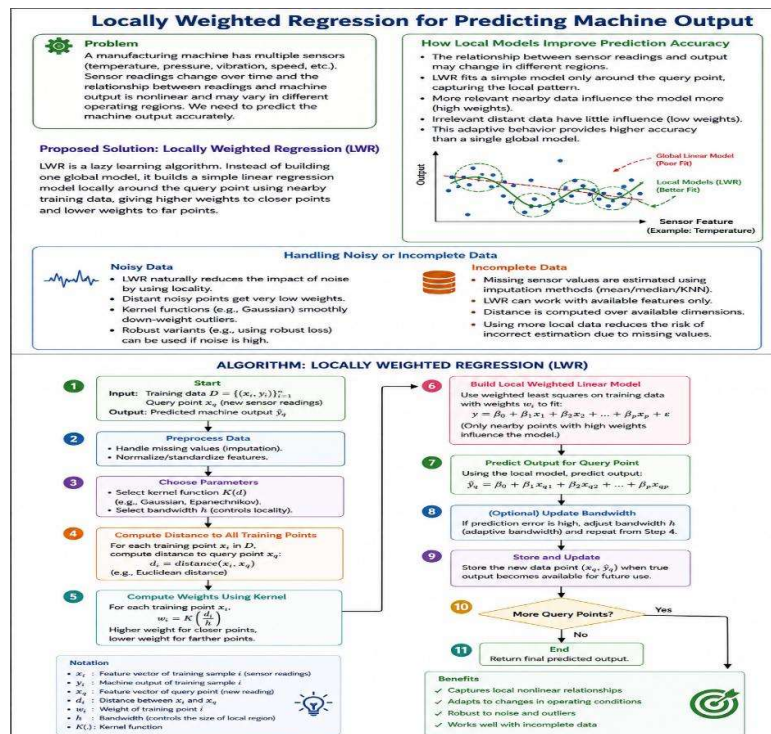
Handling Incomplete Data

If some sensor readings are missing, the system can:

- Use interpolation for short missing intervals.
- Use the most recent valid value when appropriate.
- Estimate missing sensor values using other correlated sensors.
- Ignore a sensor temporarily and calculate the prediction using available features.
- Assign lower confidence to predictions generated from incomplete data.

A data-quality component can therefore check every incoming sensor record before sending it to LWR.

Algorithm



Example

Suppose the machine has the following sensor inputs:

Sensor	Reading
Temperature	75°C
Pressure	5.2 bar
Vibration	2.1 mm/s
Speed	1500 RPM

The system searches historical records for sensor readings similar to these values. Records that are very similar receive high weights. LWR then fits a small regression model using these highly relevant observations and predicts, for example, the expected machine output.

Advantages

- Better accuracy: Focuses on locally relevant data.
- Handles nonlinear behavior: Different regions can have different relationships.
- Adapts to changing machine conditions: New observations can influence nearby predictions.
- Reduces influence of irrelevant data: Distant observations receive smaller weights.
- Suitable for time-varying sensor data: Useful when machine behavior changes with operating conditions.

Conclusion

Locally Weighted Regression is an effective solution for manufacturing environments where machine output depends on sensor readings that vary over time. By building a local regression model for each prediction, the system captures changing relationships between sensor measurements and machine output more accurately than a single global model. Combining LWR with outlier detection, missing-value handling, normalization, and continuous monitoring makes the prediction system more reliable for real-world industrial applications.

3. A healthcare company aims to classify patients into risk categories using historical medical data. Suggest a model using KNN or RBF networks. Explain how the algorithm handles large datasets, feature selection, and accuracy improvement in real-world scenarios.

Answer:

Patient Risk Classification Using KNN

A healthcare company can use the K-Nearest Neighbour (KNN) algorithm to classify patients into risk categories such as Low Risk, Medium Risk, and High Risk using historical medical data. The model can use features such as age, blood pressure, cholesterol, glucose level, BMI, heart rate, and previous medical history.

Proposed Solution

The system can follow this workflow:

Historical Patient Data → Data Preprocessing → Feature Selection → KNN Model → Risk Classification → Evaluation

For a new patient, KNN compares the patient's medical features with historical patients. The K most similar patients are selected, and the risk category that occurs most frequently among those neighbours is assigned to the new patient.

Example

Suppose a new patient has:

Feature	Reading
Age	58 years
Blood Pressure	145 mmHg
Cholesterol	230 mg/dL
Glucose	150 mg/dL
BMI	29.5

The KNN model calculates the distance between this patient's features and historical patient records.

For example, with **K = 5**, the five most similar patients may have:

Neighbour Risk Category

Patient 1	High
Patient 2	High
Patient 3	Medium
Patient 4	High
Patient 5	Medium

Since High Risk occurs most frequently, the new patient is classified as High Risk.

Handling Large Datasets

KNN can become computationally expensive with very large healthcare datasets because it calculates distances between a new patient and many historical records.

To improve performance:

- Use efficient data structures such as **KD-Trees or Ball Trees**.
- Remove duplicate and irrelevant patient records.
- Reduce the number of features through feature selection.
- Use a representative subset of historical data when appropriate.
- Apply dimensionality-reduction techniques when the dataset contains many features.
- Use optimized nearest-neighbour libraries for large datasets.

Feature Selection

Not every medical feature contributes equally to risk prediction. Irrelevant features can reduce accuracy and increase computation.

Important features can be selected using:

- Correlation analysis
- Mutual information
- Chi-square testing
- Recursive Feature Elimination
- Domain knowledge from healthcare professionals

For example, features such as age, blood pressure, glucose, cholesterol, BMI, and previous medical history may provide stronger predictive information than unrelated variables.

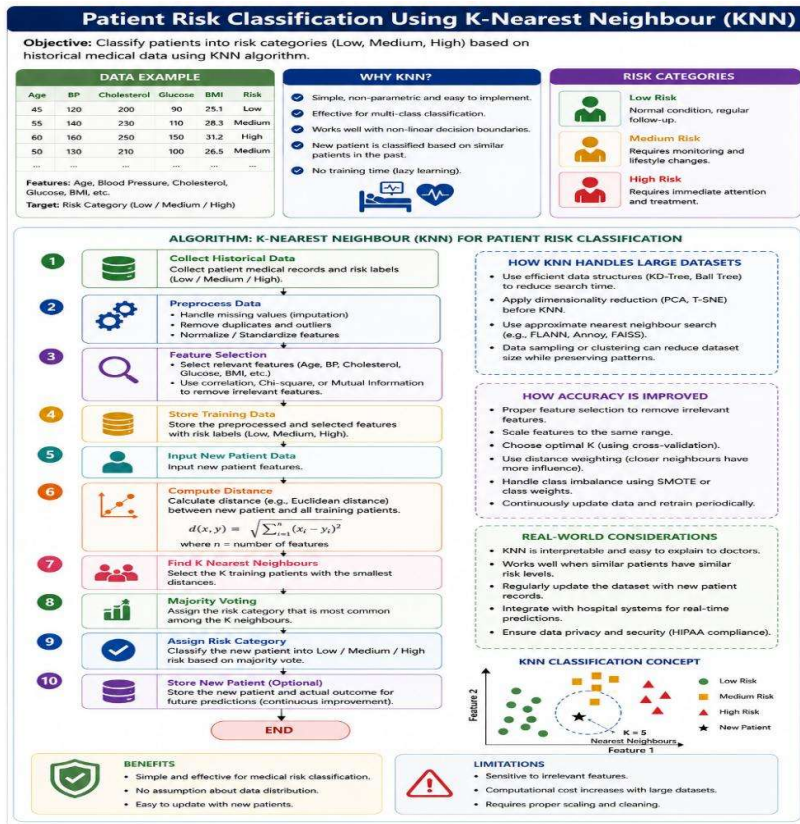
Feature selection also makes distance calculations more meaningful because KNN focuses on the most relevant patient characteristics.

Improving Accuracy

The accuracy of KNN can be improved through several techniques:

1. **Feature normalization:** Since medical features have different scales, values should be standardized before calculating distances.
2. **Optimal K selection:** Test different values of K using cross-validation and select the value providing the best performance.
3. **Distance metric selection:** Euclidean distance can be used for numerical features, while other distance measures may be suitable for different data types.
4. **Handling missing values:** Missing medical values can be handled using appropriate imputation techniques.
5. **Outlier detection:** Unusual or erroneous records should be identified before training.
6. **Cross-validation:** Evaluate the model on multiple data splits to ensure reliable performance.
7. **Class balancing:** If high-risk patients are much fewer than low-risk patients, techniques such as oversampling or class weighting can help.

Algorithm



Accuracy Evaluation

For healthcare risk classification, accuracy alone may not be sufficient. The system should also evaluate:

- Precision:** How many patients predicted as high risk are actually high risk.
- Recall:** How many actual high-risk patients are correctly identified.
- F1-score:** Balance between precision and recall.
- Confusion Matrix:** Shows correct and incorrect classifications for each risk category.

In healthcare, recall for the high-risk category is particularly important, because failing to identify a genuinely high-risk patient can have serious consequences.

Advantages

- Simple and effective:** KNN is easy to understand and implement.
- No complex training phase:** Predictions are based directly on historical cases.
- Works with multiple risk categories:** Can classify patients into low, medium, and high risk.
- Adaptable:** New validated patient records can be added to the dataset.
- Useful for case-based prediction:** Similar historical patients can provide useful reference cases.

Conclusion

KNN can provide a practical solution for classifying patients into different risk categories based on historical medical data. Feature selection and normalization improve the quality of distance calculations, while efficient nearest-neighbour methods and dimensionality reduction help handle large datasets. Cross-validation, appropriate selection of K, missing-value treatment, and class balancing can further improve accuracy.

4. An e-commerce platform wants to detect similar products for dynamic pricing strategies. Design a solution using Case-Based Learning. Discuss how similarity metrics are defined, how past cases are reused, and how the system scales with increasing product data.

Answer:

Case-Based Learning for Similar Product Detection and Dynamic Pricing

An e-commerce platform can use **Case-Based Learning (CBL)** to identify products that are similar to a newly listed product and use historical pricing cases to recommend an appropriate price. CBL works by **storing previous product cases, retrieving similar cases, adapting their solutions, and storing the new case for future use.**

Proposed Solution

The system can follow this workflow:

Product Data → Feature Extraction → Similarity Calculation → Retrieve Similar Cases → Adapt Historical Price → Recommended Price → Store New Case

Each case contains information such as:

Feature	Example
Product Category	Smartphone
Brand	Samsung
Price	₹35,000
Rating	4.5
Discount	10%
Demand	High
Sales	850 units
Stock	120 units

When a new product is added, the system searches previous cases for products with similar characteristics.

Similarity Metrics

Similarity is calculated by comparing the features of the new product with historical products.

For numerical features, **Euclidean distance** can be used:

A smaller distance indicates greater similarity.

The similarity can then be represented as:

For example, two smartphones with similar price, rating, demand, and sales history will have a high similarity score.

Different metrics can be used for different feature types:

- **Numerical features:** Euclidean or Manhattan distance
- **Categorical features:** Matching similarity
- **Text descriptions:** Cosine similarity using TF-IDF or embeddings
- **Images:** Image-feature similarity
- **Mixed data:** Weighted combination of multiple similarity measures

A weighted similarity score can be used:

where represent the importance of each feature.

Reusing Past Cases

Case-Based Learning generally follows four important steps:

1. Retrieve

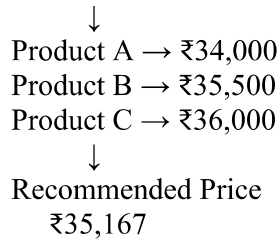
Search the case database for products that are most similar to the new product.

2. Reuse

Use the pricing information from the most similar historical products.

For example:

New Product → Similar Products



3. Revise

The recommended price can be adjusted according to current conditions such as:

- Current demand
- Inventory level
- Competitor prices
- Seasonal offers
- Customer behaviour
- Product popularity

4. Retain

After the product is sold, its actual sales performance and final price are stored as a new case. This allows the system to learn from future transactions.

Handling Increasing Product Data

As the number of products grows, comparing a new product with every stored case becomes expensive. The system can improve scalability using:

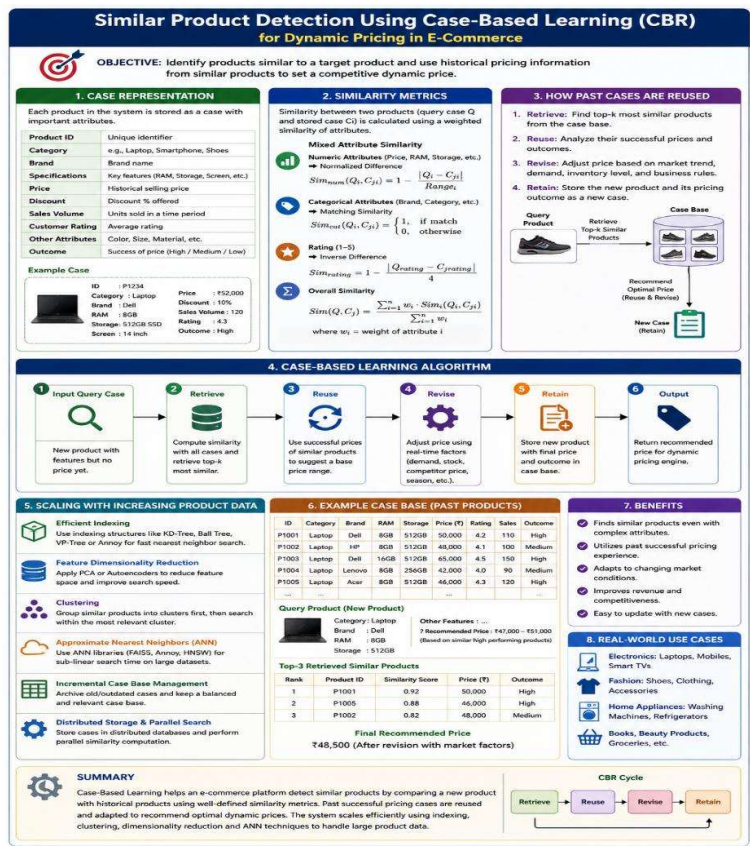
- **Indexing:** Organize cases by category, brand, or product type.
- **Nearest-neighbour search:** Retrieve only the most relevant cases.
- **Clustering:** Group similar products into clusters and search within the appropriate cluster.
- **Vector databases:** Store product embeddings and perform fast similarity searches.
- **Feature reduction:** Remove unnecessary features to reduce computation.
- **Caching:** Store frequently requested similarity results.
- **Distributed storage:** Divide large case databases across multiple servers.

For example, instead of searching through **10 million products**, the system can first identify the product category and search only within the relevant product cluster.

Advantages

- **Learns from historical experience:** Uses previous successful pricing cases.
- **Adapts to market conditions:** Historical solutions can be modified according to current demand and inventory.
- **Easy to update:** New transactions can be stored as new cases.
- **Handles different product types:** Similarity can combine numerical, categorical, text, and image features.
- **Supports dynamic pricing:** Historical prices provide a starting point that can be adjusted using real-time conditions.
- **Scalable:** Indexing, clustering, and vector search can make retrieval efficient for large product catalogs.

Algorithm



Conclusion

Case-Based Learning provides an effective approach for detecting similar products and supporting dynamic pricing in e-commerce. The system retrieves similar historical products, reuses their pricing experience, adapts the recommendation to current market conditions, and retains the new case for future decisions. By combining weighted similarity metrics with indexing, clustering, and efficient nearest-neighbour or vector search, the system can continue to operate effectively as the product database grows.

5. A logistics company needs to forecast delivery times based on location and traffic patterns. Propose a solution using Radial Basis Function networks or Locally Weighted Regression. Explain how the model captures non-linear relationships and improves prediction reliability under varying conditions.

Answer:

Delivery Time Forecasting Using Radial Basis Function Network

A logistics company can use a Radial Basis Function (RBF) Network to forecast delivery times based on factors such as location, distance, traffic conditions, weather, time of day, vehicle type, and historical delivery performance.

Proposed Solution

The system can follow this workflow:

GPS & Traffic Data → Data Preprocessing → Feature Selection → RBF Network → Predicted Delivery Time → Monitoring

Feature	Example
Distance	25 km
Traffic Level	High
Average Speed	28 km/h
Time of Day	6:30 PM
Weather	Rainy
Route Type	Urban
Historical Delay	12 min

The output of the RBF network is the estimated delivery time, such as 65 minutes.

How RBF Networks Capture Non-Linear Relationships

A linear model assumes that delivery time changes proportionally with factors such as distance and traffic. However, real-world delivery conditions are often nonlinear.

For example:

- A small increase in traffic may cause a small delay under normal conditions.
- During rush hour, the same increase may cause a much larger delay.
- Rain combined with heavy traffic can produce an even greater delay.
- Different locations can have completely different traffic patterns.

An RBF network handles these relationships using radial basis functions in its hidden layer.

Each hidden neuron represents a particular region or pattern in the input data.

where is the current delivery condition, is the center of an RBF neuron, and controls the width of the function.

Conditions similar to a neuron's center produce a high activation, while different conditions produce a lower activation.

Example

Suppose historical delivery data contains:

Traffic	Weather	Distance	Delivery Time
Low	Clear	20 km	35 min
Medium	Clear	20 km	45 min
High	Clear	20 km	60 min
High	Rain	20 km	75 min

For a new delivery with high traffic, rainy weather, and 20 km distance, the RBF network identifies similar historical situations and uses their patterns to estimate the delivery time.

Improving Prediction Reliability

Traffic variation: Real-time traffic information allows the prediction to change when congestion increases or decreases.

Location differences: Different RBF centers can represent different geographical and traffic patterns.

Weather conditions: Rain and other weather conditions can be included as features to account for their effect on travel time.

Time-dependent behavior: Rush-hour and non-rush-hour patterns can be learned separately.

Outlier handling: Incorrect GPS readings and unusual delivery records can be detected and removed or given less influence during training.

Continuous learning: New delivery results can be added periodically so that the model adapts to changing traffic and road conditions.

Handling Different Conditions

Normal Traffic + Clear Weather



Short Delivery Time

Heavy Traffic + Clear Weather



Moderate Delay

Heavy Traffic + Rain



Large Delay

Rush Hour + Heavy Traffic + Rain



Very Large Delay

This local pattern learning enables the RBF network to model complex relationships more effectively than a simple linear regression model.

Advantages

- Captures nonlinear relationships: Learns complex interactions between distance, traffic, weather, and location.
- Adapts to different regions: Local RBF centers can represent different traffic patterns.
- Handles changing conditions: New traffic and delivery data can be incorporated into future training.
- Fast prediction: Once trained, the network can generate predictions quickly.
- Improved reliability: Combining multiple relevant local patterns reduces dependence on a single global relationship.

Algorithm:



Conclusion

An RBF Network is well suited for logistics delivery-time forecasting because delivery time depends on complex and nonlinear combinations of location, distance, traffic, weather, and time. The hidden RBF layer captures local patterns, while the output layer combines these patterns to generate the final prediction. By using real-time traffic information, preprocessing noisy data, selecting relevant features, and periodically retraining with new delivery records, the system can provide more reliable delivery-time predictions under changing real-world conditions.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	25	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand